

CHM Platform Roadmap — Dev, Cognito & MediaHub

Prepared for: CHM leadership, CHT platform team, MediaHub engineering

Updated: June 16, 2026

Status: Phase 1 complete (staging destroyed). Phase 2 in progress — Cognito prod; local dev only until Cognito is stable, then scoped AWS dev deploy.

Executive Summary

This plan consolidates architecture and sequencing decisions for the CHM platform:

1. **~~Destroy staging~~ — done.**
2. **Cognito in production** — migrate existing GoTrue users; cut over prod auth.
3. **Cognito dev pool + specs in parallel with prod** — provision **prod and dev Cognito** (Terraform, app clients, groups) in the **same Phase 2 workstream**. Use dev pool for local/testing; **do not fully deploy AWS dev** (ECS/RDS/CI) until Phase 3.
4. **After Cognito prod is stable, fully deploy** the hosted dev environment (AWS stack + deploy pipeline). ECS/RDS/domain scope deferred to Phase 3.
4. **Move MediaHub** into the CHT AWS footprint as separate microservices (separate repo, deploys, RDS).
5. **CHT-only Redis cache** (24h TTL) with catalog refresh on sync.

Total estimated timeline: 3–4 months.

Architecture Principles

Decision	Choice
Auth	CHT-owned Cognito + required TOTP MFA; end users never authenticate on MediaHub
Users	Migrate existing GoTrue users; remap <code>User . authId</code> in CHT Postgres — no greenfield accounts
Environments	Prod (Cognito cutover first) → full AWS dev deploy (Phase 3) + local — staging destroyed
Cognito	Prod + dev pools provisioned together in Phase 2 Terraform; prod cutover in Phase 2; dev pool used for testing
Deployed dev (ECS/RDS/CI)	Phase 3 only — after Cognito prod stable; sizing/domain scoped later

Service	Tasks (typical)	Database
chm-backend	2–3	CHT RDS
chm-worker	1–2	CHT RDS
mediahub-api	2	MediaHub RDS
mediahub-worker	1	MediaHub RDS
mediahub-reports	1 (+ SQS scale)	MediaHub RDS

MediaHub RDS (production)

Setting	Value
Engine	PostgreSQL 15.17
Instance	db.t3.small
Storage	50 GB gp3 (grow as needed)
Multi-AZ	Yes
Backup retention	30 days
Access	mediahub-api, mediahub-worker, mediahub-reports SGs only

CHT-Only Cache Strategy

Scope: Cache public catalog and KOL network reads on chm-backend only. No HTTP cache in MediaHub. No FastAPI middleware.

TTL: 86400 seconds (24 hours) as safety net.

Refresh on sync: When mediahub-worker completes a successful sync, call CHT internal endpoint to clear catalog cache keys. Next user request repopulates from MediaHub with new content.

```
Normal: User → chm-backend → Redis hit → return (up to 24h)
Miss: User → chm-backend → mediahub-api → store in Redis EX 86400
After sync:
mediahub-worker → POST /internal/cache/catalog/clear
chm-backend → DEL cht:catalog:* and cht:kol-network:*
Next request → cache miss → fresh data → cache 24h again
```

Redis key patterns:

Key	Cached
cht:catalog:tags	Yes
cht:catalog:clips:{hash(params)}	Yes
cht:catalog:playlists:{hash}	Yes
cht:kol-network:{hash}	Yes
Auth, payments, admin, writes	Never

Infrastructure: One ElastiCache cluster (cache.t3.micro or small) with key prefixes; chm-backend reads/writes catalog keys; worker triggers clear via internal secret + SG-restricted endpoint.

Tracing (Request ID)

Phase in during MediaHub migration (Phase 5):

- Generate or preserve `X-Request-Id` at chm-backend (ALB or NestJS middleware).
- Forward to mediahub-api on every internal call.
- Log JSON with `request_id` in CloudWatch on both sides.
- Pass `request_id` in SQS message attributes for report jobs.
- Optional later: OpenTelemetry → AWS X-Ray (not App Mesh).

Phase 0 — Spec & Runbooks (1–2 weeks)

Goal: Signed documentation before infra changes.

Deliverables (drafts in [docs/runbooks/](../runbooks/README.md)):

- [Cognito migration spec](../runbooks/cognito-migration-spec.md) — sign-off table
- [User migration runbook](../runbooks/cognito-user-migration.md) — GoTrue → Cognito, authId remap
- [Secrets migration](../runbooks/secrets-migration-staging-to-prod.md) — before staging destroy
- [Staging teardown](../runbooks/staging-teardown.md)
- [Local Cognito setup](../runbooks/local-cognito-setup.md) — Phase 2 build
- [Cognito prod cutover](../runbooks/cognito-prod-cutover.md) + [MediaHub auth checklist](../runbooks/mediahub-auth-decommission-checklist.md)
- [Stable dev spec](../runbooks/stable-dev-environment.md) — Phase 3 (after Cognito prod)
- [MediaHub platform cutover](../runbooks/mediahub-platform-cutover.md) — Phase 4 outline
- [Cache & sync contract](../runbooks/cache-sync-contract.md) — Phase 4

Exit criteria: Runbooks approved in [runbooks/README.md](../runbooks/README.md) sign-off table; rollback defined for each phase.

Phase 1 — Destroy Staging ■ Complete

Goal: Decommission staging AWS stack before Cognito work.

Completed:

- Staging secrets reconciled to prod (see [secrets migration runbook](../runbooks/secrets-migration-staging-to-prod.md))
- terraform destroy on us-east-1-staging
- Retire deploy-staging.yml or archive workflow (follow-up PR)
- Remove GitHub staging environment (after secret names documented)
- Update deployment.md, incident-response.md, architecture.md
- Retire DNS: staging.testapp.communityhealth.media (if still delegated)

Until Phase 3: daily development = **local only** (docker-compose + local Cognito). No hosted non-prod environment.

Phase 2 — Cognito on Production (4–7 weeks)

Goal: Production auth on Cognito; existing users migrated; GoTrue decommissioned for end users.

2a — Local + dev Cognito during build (parallel)

Implement auth against **docker-compose locally**, using the **AWS dev Cognito pool** provisioned in Step 2b (or cognito-local until pool exists). **No full AWS dev stack** (ECS/RDS) in Phase 2.

Option	Use case
Dev Cognito pool (cht-platform-dev)	Primary — same spec as prod; test signup/MFA/OAuth before prod cutover
cognito-local / LocalStack	Optional offline fallback
VITE_USE_DEV_AUTH=true	Non-auth UI work only

```
localhost:5173 → localhost:3000 → local Postgres
↓
dev Cognito pool (Phase 2 Terraform – NOT prod pool)
```

Pre-cutover validation: auth flows on **dev pool** + local backend; dry-run migration on prod DB snapshot.

2b — Cognito infra — prod + dev together (week 1–2)

Provision **both pools in one Terraform pass** (same spec, different pools):

Pool	Name	Purpose
Prod	cht-platform-prod	Production cutover + real users
Dev	cht-platform-dev	Local/testing; ready for Phase 3 hosted dev

Per pool (prod and dev):

- App client `cht-web` (PKCE); dev pool includes localhost callback URLs
- App client `mediahub-admin` (CHM operators)
- Groups: `cht-kol`, `cht-hcp`, `cht-pharma-client`
- MFA: required TOTP (grace period on **prod** only — `_TBD_`)

Secrets Manager:

- Prod Cognito IDs → prod secrets (used at prod cutover)
- Dev Cognito IDs → dev secrets file / team store (for local `.env`)
- Retain GoTrue secrets until **prod** cutover day

Not in Phase 2: ECS, RDS, ALB, or deploy workflow for hosted dev — that is Phase 3.

2c — Application (week 2–5)

- Backend: Cognito SignUp, InitiateAuth, MFA challenge, JWKS validation
- Remove GoTrue proxy from `auth.controller.ts`
- Frontend: Cognito on CHT domain only; remove MediaHub OAuth redirects
- Keep Postgres sessions + httpOnly cookies
- Chatbot: Cognito token via backend
- **Unchanged:** MediaHubService, MediaHubSyncService (API key only)

Validate on dev Cognito pool + local docker before prod cutover.

2d — User migration — no new users (week 5–6)

1. Export GoTrue users: email, sub, `user_metadata` (NPI, profession, institution).
2. Import into Cognito prod via bulk import or User Migration Lambda.
3. Remap CHT Postgres `User.authId`: GoTrue sub → Cognito sub (match by email).
4. OAuth users: federate Google in Cognito; link by email to existing User row.
5. Communicate: same account, one-time MFA setup — not a new registration.

Pre-cutover validation:

- Auth flows proven on **dev Cognito pool** + local backend

- Dry-run import counts match GoTrue export
- Dry-run authId remap on prod DB snapshot (read-only)
- Rollback plan: keep GoTrue secrets 48h post-cutover

2e — MediaHub auth coordination (parallel)

- Block end-user OAuth redirects to MediaHub
- Deactivate non-admin GoTrue users (learners/HCP/KOL) — content data unchanged
- Confirm API key valid through and after cutover

2f — Production cutover (week 6–7)

- Maintenance window + user comms
- Import users to Cognito prod; run authId remap
- Deploy Cognito-enabled backend + frontend
- MediaHub: GoTrue off for end users
- Monitor login, MFA, catalog, HCP upsert, payments
- Remove GOTRUE_JWT_SECRET, SUPABASE_* from prod secrets (after 48h rollback window)
- Update subprocessor register

Exit criteria:

- Cognito prod **up and running**; existing users log in with same email
- No duplicate User rows created
- End users never touch MediaHub auth
- Catalog still works via API key (external MediaHub URL until Phase 4)

Phase 3 — Full AWS dev deploy (after Cognito prod stable)

Goal: Deploy the **hosted dev stack** (ECS, RDS, ALB, S3, CI/CD) — staging replacement. **Cognito dev pool already exists from Phase 2**; connect it when the stack goes live.

Scope: ECS/RDS/domain/CI sizing **deferred** until Phase 3 kickoff — does not block Phase 2.

Prerequisites:

- Phase 2 exit criteria met (Cognito **prod** stable $\geq$ 1–2 weeks)
- Phase 2b complete (Cognito **dev** pool already provisioned)
- Phase 1 follow-up complete

Planning inputs (TBD at Phase 3 kickoff):

Topic	Status
Cognito dev pool	■ Provisioned in Phase 2 — wire dev ECS to existing pool

Topic	Status
Domain	_TBD_ (e.g. dev.testapp.communityhealth.media)
Terraform	_TBD_ (us-east-1-dev ECS/RDS/ALB)
GitHub dev env + deploy workflow	_TBD_
ECS/RDS sizing	_TBD_
MediaHub	External API until Phase 4

Local dev (ongoing):

docker-compose + **dev Cognito pool** remains valid for day-to-day work even after hosted dev exists.

Exit criteria (define when Phase 3 is scoped):

- Deployed dev URL live, using existing dev Cognito pool
- Engineers can deploy a branch to dev without touching prod
- Prod remains the only customer-facing environment

See [stable-dev-environment.md](../runbooks/stable-dev-environment.md).

Phase 4 — MediaHub Platform Move (6–10 weeks)

Goal: mediahub-api, mediahub-worker, mediahub-reports on CHT ECS cluster; separate RDS; internal integration.

4a — Repository & CI (week 1–2)

- Create mediahub-platform repo (Python / FastAPI)
- Separate Dockerfiles and deploy workflows per service
- Publish OpenAPI for /api/public/*
- Contract tests consumable by CHT CI

4b — Production infra (week 2–4)

Add to us-east-1 Terraform:

- RDS module: mediahub-db (Postgres 15.17, db.t3.small, Multi-AZ, 50 GB)
- ElastiCache for CHT catalog cache (+ optional Hub locks)
- ECS: mediahub-api (×2), mediahub-worker (×1), mediahub-reports (×1)
- SQS report queue + DLQ
- S3 buckets for reports/assets
- Secrets: mediahub/database, API keys, LLM keys

- Security groups: Hub DB accessible only from Hub services

4c — Data migration (week 4–5)

- pg_dump from MediaHub EC2 Postgres
- Restore to mediahub-db
- Validate clips, KOLs, tags, analytics row counts
- Point mediahub-api at new RDS
- No new auth users — identity already on Cognito from Phase 2

4d — Cutover (week 5–6)

- Deploy Hub services to prod ECS (same cluster as chm-backend)
- Update CHT MEDIAHUB_BASE_URL to internal service URL
- Integration tests: catalog, KOL network, HCP upsert
- Decommission MediaHub EC2 Compose stack
- Optional: admin hostname to new mediahub-api

4e — CHT cache + sync refresh (week 6–7)

- Wire ElastiCache to chm-backend
- NestJS cache on catalog/KOL routes (24h TTL)
- Implement POST /internal/cache/catalog/clear
- mediahub-worker calls clear after successful sync
- Verify new clips appear on next request after sync

Exit criteria:

- Five ECS services deploy independently
- EC2 MediaHub retired
- CHT calls Hub internally; API key auth unchanged
- Cache refreshes on sync

Phase 5 — Hardening (ongoing)

- X-Request-Id across chm-backend ↔ mediahub-api ↔ SQS jobs
- CloudWatch alarms: Hub ECS, both RDS instances, Redis, DLQs
- DR drill: restore mediahub-db from snapshot
- Performance Insights: upgrade MediaHub RDS if CPU sustained > 60%
- Auth + catalog contract tests in CI
- Optional: OpenTelemetry / X-Ray

Dependency Order

Phase 0: Specs & runbooks
↓
Phase 1: Destroy staging
↓
Phase 2: Cognito prod + dev pools (same Terraform); prod cutover; local testing on dev pool
↓
Phase 3: Full AWS dev deploy (ECS/RDS/CI) – Cognito dev pool already exists
↓
Phase 4: MediaHub ECS + separate RDS + cache
↓
Phase 5: Hardening

Destroy staging before Cognito — no dev env blocker.

Phase 2: Prod + dev **Cognito** together; prod cutover; no hosted dev stack yet.

Phase 3: Full **AWS dev deploy** only after Cognito prod is stable (Cognito dev pool already provisioned).

Timeline Overview

Phase	Duration	Outcome
0 — Spec	1–2 weeks	Signed runbooks
1 — Destroy staging	■ Done	Staging gone; cost saved
2 — Cognito	4–7 weeks (in progress)	Prod + dev pools; prod cutover; test on dev pool locally
3 — Full AWS dev deploy	TBD (after Cognito prod stable)	ECS/RDS/CI; dev pool already exists
4 — MediaHub move	6–10 weeks	Microservices on platform; internal URL
5 — Hardening	Ongoing	Tracing, DR, monitoring

Total: approximately 3–4 months.

Cost Estimate

Change	Approx. monthly impact
Destroy staging	Save ~\$50–100

Change	Approx. monthly impact
Cognito prod	~\$0–5 (MAU-based)
AWS dev (Phase 3, scoped later)	~\$40–80 (estimate — confirm when scoped)
MediaHub RDS (db.t3.small Multi-AZ)	+\$55–75
ElastiCache (cache.t3.micro)	+\$12–25
MediaHub Fargate (api×2, worker×1, reports×1)	+~\$72
LLM API (reports)	Variable

Risks & Mitigations

Risk	Mitigation
No hosted dev stack until Phase 3	Dev Cognito pool in Phase 2; local docker for app work; strict prod cutover checklist
Deployed dev scope unknown	Phase 3 planning session after Cognito stable; do not delay Cognito
User migration mismatch	Email-based remap; reconciliation report before cutover
MFA adoption friction	Clear comms: same account, MFA enrollment once
MediaHub cutover breaks catalog	Parallel run EC2 + ECS until validated; API key unchanged
Cache stale after sync	Worker clears catalog keys; 24h TTL as backstop

Immediate Next Actions

1. Close Phase 1 follow-up: archive staging workflow, update docs, retire GitHub staging env.
2. Lock open decisions in [cognito-migration-spec.md](../runbooks/cognito-migration-spec.md) (MFA grace, login UI, cutover date).
3. **Phase 2:** Terraform **prod + dev Cognito pools** together; app auth; validate on dev pool locally; prod cutover.
4. **Defer Phase 3** (full AWS dev deploy) until Cognito prod is stable ~1–2 weeks.
5. MediaHub: [mediahub-auth-decommission-checklist.md](../runbooks/mediahub-auth-decommission-checklist.md) + GoTrue user export.

Related Documents

- CHT-MediaHub-Go-Forward-Options.md
- CHT-Auth-Decoupling-Next-Steps-Report.md
- CHT-Platform-Assessment-Report.md

Prepared by: Uche Aduakaa

Reviewed by: Adaze Oviawe

Approved: June 16, 2026 at 08:28 PM EDT